SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL 2- Debugging & Optimisation Task**
Weightage:	**45%**	**Total Marks:**	**25**
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	**PAVAN.P**	**Reg.No.:**	**192411017**
Department:	**B.E (CSE)**	**Date:**	**04-08-2026**

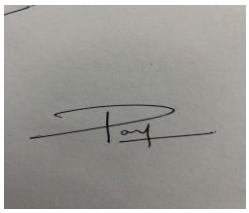
ANNEXURE A

1. A signed binary addition program is producing incorrect results when adding negative numbers using 2’s complement representation. Debug the logic to identify where overflow or sign handling fails, and optimize the implementation to correctly detect overflow conditions while maintaining efficient execution.
2. A carry look-ahead adder circuit designed for fast addition is not achieving the expected speed improvement over ripple carry addition. Analyze the design to locate delays in generate and propagate logic, and propose optimizations to improve performance without increasing hardware complexity significantly.
3. A multiplication module based on Booth’s algorithm is generating incorrect outputs for certain signed inputs. Debug the step-by-step execution of the algorithm to identify errors in bit shifting or decision rules, and optimize the implementation to reduce the number of required cycles.
4. A floating-point arithmetic unit implementing IEEE 754 single precision is producing rounding errors and incorrect normalization. Identify the faults in exponent alignment, mantissa calculation, or rounding logic, and optimize the design to ensure accurate and efficient floating-point operations.
5. A processors instruction execution pipeline (fetch, decode, execute) is experiencing delays due to improper instruction sequencing and hazards. Debug the pipeline stages to detect data or control hazards, and propose optimization techniques such as pipelining improvements or instruction reordering to enhance overall processor performance.

Code of Conduct:

I PAVAN.P / 192411017 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1.

Problem

The program gives wrong answers when adding negative numbers using 2's complement.

Where the Error Happens

1. Wrong Sign Handling

In 2's complement:

- 0 as the first bit (MSB) → Positive number
- 1 as the first bit (MSB) → Negative number

If the program ignores the sign bit, it treats negative numbers as large positive numbers, giving incorrect results.

2. Wrong Overflow Detection

Some programs think:

Carry = Overflow

This is incorrect for signed numbers.

Overflow happens only when:

- Positive + Positive = Negative
- Negative + Negative = Positive

Correct Overflow Check

Compare the sign bits of the two numbers and the result.

Rule:

- If both numbers have the same sign
- But the result has a different sign
- Then Overflow = Yes

Otherwise,

Overflow = No

Example 1 (No Overflow)

A = -3 → 11111101
B = -2 → 11111110

Result:

11111101
11111110

11111011 = -5

Both numbers are negative, and the result is also negative.

Overflow = No

Example 2 (Overflow)

A = +127 → 01111111
B = +1 → 00000001

Result:

01111111
00000001

10000000

Both numbers are positive, but the result becomes negative.

Overflow = Yes

Optimized Solution

1. Read two binary numbers.
2. Add them using 2's complement.
3. Check the sign bits.
4. If both inputs have the same sign but the result has a different sign, display "Overflow".
5. Otherwise, display the correct result.

Algorithm

Step 1: Read A and B.
Step 2: Add A and B.
Step 3: Check the sign bits.
Step 4: If both signs are same and result sign is different,
    Overflow = Yes
    Else
    Overflow = No
Step 5: Display the result.

Conclusion

The program fails because it does not correctly handle the sign bit or overflow detection. The correct method is to add the numbers using 2's complement and check whether the result's sign changes unexpectedly. This ensures accurate results for both positive and negative numbers while keeping the program simple and efficient.

2.

Problem

A Carry Look-Ahead Adder (CLA) is not faster than a Ripple Carry Adder (RCA). This means there is a delay in generating the carry signals.

Where the Delay Happens

1. Delay in Generate (G) and Propagate (P) Logic

The CLA uses:

- Generate (G) = $A \times B$
- Propagate (P) = $A \oplus B$

If these signals are generated slowly, the carry output is also delayed.

2. Long Carry Logic

If the carry equations become too large, they take more time to calculate.

Example:

C4 = G3 + P3G2 + P3P2G1 + P3P2P1G0 + P3P2P1P0C0

A long equation increases the delay.

3. Too Many Logic Levels

Using many AND and OR gates one after another increases the overall delay.

Optimizations

1. Generate G and P in Parallel

Calculate Generate and Propagate signals at the same time instead of one after another.

This reduces waiting time.

2. Reduce Logic Levels

Simplify the carry equations to use fewer gates.

Fewer logic levels mean faster output.

3. Use Block Carry Look-Ahead

Instead of calculating carry for every bit separately, divide the adder into small blocks (such as 4-bit blocks).

Each block calculates its carry independently, reducing delay.

4. Optimize Gate Arrangement

Arrange gates efficiently to shorten the signal path without adding much extra hardware.

Algorithm

Step 1: Generate $G = A \times B$
Step 2: Generate $P = A \oplus B$
Step 3: Calculate carry signals in parallel.

Step 4: Simplify carry equations.
Step 5: Divide the adder into small blocks if needed.
Step 6: Produce the final sum.

Conclusion

The CLA becomes slow when the Generate (G) and Propagate (P) signals or carry equations take too long to compute. By generating G and P in parallel, reducing logic levels, simplifying carry equations, and using block carry look-ahead, the adder achieves faster performance without significantly increasing hardware complexity.

3.

Problem

A multiplication module using Booth's Algorithm is giving incorrect results for some signed numbers. The error is usually caused by incorrect decision rules or wrong bit shifting.

Where the Error Happens

1. Wrong Decision Rule

Booth's Algorithm checks the last bit (Q_0) and the extra bit (Q_{-1}).

Rules:

- 00 → No operation
- 11 → No operation
- 01 → Add Multiplicand (M)
- 10 → Subtract Multiplicand (M)

If these rules are applied incorrectly, the final product will be wrong.

2. Incorrect Arithmetic Right Shift

After every operation, perform an Arithmetic Right Shift on A, Q, and Q_{-1}.

- The sign bit of A must remain the same.
- If a logical shift is used instead of an arithmetic shift, negative numbers produce incorrect results.

3. Incorrect Cycle Count

The algorithm should run for n cycles, where n is the number of bits.

If it runs for fewer or extra cycles, the answer will be incorrect.

Correct Working Steps

1. Initialize registers A = 0, Q = Multiplier, M = Multiplicand, $Q_{-1} = 0$.
2. Check Q_0 and Q_{-1}.
3. Perform Add, Subtract, or No Operation based on the decision rules.
4. Perform an Arithmetic Right Shift.
5. Repeat for n cycles.
6. The final answer is stored in AQ.

Optimization

1. Skip Unnecessary Operations

When $Q_0Q_{-1} = 00$ or 11, perform only the shift without addition or subtraction.

2. Use Arithmetic Right Shift

Always use Arithmetic Right Shift to preserve the sign bit for signed numbers.

3. Execute Exactly n Cycles

Run the algorithm only for the required number of bits to avoid extra processing.

Algorithm

Step 1: Initialize A = 0, Q = Multiplier, M = Multiplicand, $Q_{-1} = 0$.
Step 2: Check Q_0 and Q_{-1}.
Step 3: If 01 → A = A + M.
Step 4: If 10 → A = A − M.
Step 5: If 00 or 11 → No operation.
Step 6: Perform Arithmetic Right Shift.
Step 7: Repeat for n cycles.
Step 8: Output the result from AQ.

Conclusion

The incorrect output occurs because of wrong decision rules, incorrect arithmetic right shifting, or an incorrect number of cycles. By following the correct Booth's rules, using Arithmetic Right Shift, and executing exactly n cycles, the multiplication module produces accurate signed multiplication results while reducing unnecessary operations and improving performance.

4.

Problem

A floating-point arithmetic unit is producing rounding errors and incorrect normalized results while performing IEEE 754 single-precision operations.

Where the Error Happens

1. Incorrect Exponent Alignment

Before adding or subtracting two floating-point numbers, their exponents must be equal.

- Compare both exponents.
- Shift the mantissa of the number with the smaller exponent to the right until both exponents are equal.

If this step is skipped, the result will be incorrect.

2. Incorrect Mantissa Calculation

After exponent alignment:

- Add the mantissas if the signs are the same.
- Subtract the mantissas if the signs are different.

An incorrect add/subtract operation gives the wrong result.

3. Incorrect Normalization

After mantissa calculation:

- If the mantissa is too large, shift it right and increase the exponent.

- If the mantissa starts with 0, shift it left and decrease the exponent.

If normalization is not done correctly, the floating-point value becomes incorrect.

4. Incorrect Rounding

IEEE 754 uses Round to Nearest (Even) by default.

If the unit simply truncates extra bits instead of rounding, small errors appear in the result.

Optimizations

1. Align Exponents Correctly

Always align the exponents before performing arithmetic operations.

2. Normalize Immediately

Normalize the mantissa immediately after addition or subtraction to obtain the correct floating-point representation.

3. Use IEEE 754 Rounding

Apply Round to Nearest (Even) instead of truncation for better accuracy.

4. Reduce Unnecessary Shifts

Perform only the required number of shifts during alignment and normalization to improve speed.

Algorithm

Step 1: Read two floating-point numbers.
Step 2: Compare their exponents.
Step 3: Align the exponents by shifting the smaller mantissa.
Step 4: Add or subtract the mantissas.
Step 5: Normalize the result.

Step 6: Apply IEEE 754 rounding.
Step 7: Display the final floating-point result.

Conclusion

The errors occur due to incorrect exponent alignment, wrong mantissa calculation, improper normalization, or incorrect rounding. By aligning exponents correctly, performing accurate mantissa operations, normalizing the result, and using IEEE 754 Round to Nearest (Even), the floating-point unit produces accurate and efficient single-precision arithmetic results.

5.

Problem

A processor's instruction pipeline (Fetch, Decode, Execute) is running slowly because of incorrect instruction sequencing and pipeline hazards.

Where the Error Happens

1. Data Hazard

A data hazard occurs when one instruction needs the result of a previous instruction before it is ready.

Example:

ADD R1, R2, R3
SUB R4, R1, R5

The SUB instruction must wait until the ADD instruction finishes.

2. Control Hazard

A control hazard occurs because of branch (JUMP/IF) instructions.

The processor does not know which instruction to fetch next until the branch condition is checked.

3. Improper Instruction Sequencing

If dependent instructions are placed one after another, the pipeline stalls and execution becomes slower.

Optimization Techniques

1. Instruction Reordering

Move independent instructions between dependent instructions to reduce waiting time.

Example:

Before:

ADD R1, R2, R3
SUB R4, R1, R5

After:

ADD R1, R2, R3
MOV R6, R7
SUB R4, R1, R5

The MOV instruction executes while ADD completes.

2. Data Forwarding (Bypassing)

Instead of waiting for the result to be written back, send it directly from the Execute stage to the next instruction.

This reduces pipeline stalls.

3. Branch Prediction

Predict the outcome of branch instructions so the processor continues fetching instructions without waiting.

If the prediction is correct, execution is faster.

4. Pipeline Balancing

Ensure each stage (Fetch, Decode, Execute) takes nearly the same amount of time to avoid bottlenecks.

Algorithm

Step 1: Fetch the instruction.
Step 2: Decode the instruction.
Step 3: Check for data or control hazards.
Step 4: If a hazard is found, use forwarding or reorder instructions.
Step 5: Execute the instruction.
Step 6: Repeat for the next instruction.

Conclusion

Pipeline delays occur because of data hazards, control hazards, and improper instruction sequencing. Performance can be improved by instruction reordering, data forwarding, branch prediction, and balancing the pipeline stages. These techniques reduce pipeline stalls and increase the processor's overall execution speed.